

PMEncrypter

Documentación

Introducción

- ¡Bienvenido a la documentación de PMEncrypter! Este documento tiene la intención de informar, mostrar y presentar el proceso de documentación y desarrollo el cual ha pasado la aplicación antes mencionada, desde sus usos intencionados, hasta el código que posee la aplicación por dentro.

PMEncrypter ha sido creado en base a código en el lenguaje C#, más que nada debido a la compatibilidad y facilidad de codificar en aquel lenguaje, con la ayuda de ChatGPT para arreglar ciertos problemas y bugs que tenía el código.

PMEncrypter tiene la función de calcular el *hash* de un archivo usando el algoritmo **SHA-256**, sin embargo, no se debe malentender que la aplicación *cifré* archivos, sólo *calcula* los hashes que posee el archivo en específico, pero fuera de eso, no realiza ningún otro cambio o alguna acción adicional aparte de la función de su código único de 63 caracteres.

Proceso de creación el Software

En un inicio elegimos el tipo de algoritmo SHA-256, utilizando el lenguaje de C# para el proyecto a través de visual studio code, además de contar con dotnet para iniciar formando la estructuras de nuestras carpetas.

Luego creamos el archivo donde vivirá nuestro algoritmo y su respectivo de dotnet.

Al completar estos pasos compilamos y tratamos de ejecutar nuestro programa en la terminal para verificar su correcta creación:

```
PMEncrypter v1.0.0
```

```
Herramienta de cálculo de hash SHA-256
```

```
Copyright 2024 - Proyecto de Empaquetado
```

```
PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter>
```

```
echo "archivo de prueba" > test.txt
```

```
>> dotnet run -- hash test.txt
```

```
=== PMEncrypter - Herramienta de Hash SHA-256 ===
```

```
Archivo: test.txt
```

```
Tamaño: 62 bytes
```

```
SHA-256: 4c00aa3ed3b1f66c07e25bb3518f1c5cdf103d1c42db207a88a0716470f98419
```

```
Ruta: C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter\test.txt
```

```
PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter>
```

Luego de esto generamos las carpetas y archivo ejecutable .MSI gracias a WiX Toolset

(! Acá tuvimos un problema con candle ya que no parecía haberse ubicado correctamente, pero

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter> candle -?
⊙ candle : El término 'candle' no se reconoce como nombre de un cmdlet,
función, archivo de script o programa ejecutable. Compruebe si escribió
correctamente el nombre o, si incluyó una ruta de acceso, compruebe que
dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ candle -?
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (candle:String) [], CommandNo
tFoundException
+ FullyQualifiedErrorId : CommandNotFoundExcep
```

luego de ejecutar " "C:\Program Files (x86)\WiX Toolset v3.14\bin\candle.exe" con su ruta completa pudo funcionar correctamente.

Luego creamos el archivo Product.wxs para generar nuestro ejecutable

(!) En el proceso tuve un problema por poseer un par de acentos en el código, dándome error, pero luego de caer en cuenta del mismo fue fácil corregirlo.

```
● & "C:\Program Files (x86)\WiX Toolset v3.14\bin\candle.exe" Product.wxs
>>
Windows Installer XML Toolset Compiler version 3.14.1.8722
Copyright (c) .NET Foundation and contributors. All rights reserved.

Product.wxs
PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter>

⊙ PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter>
⊙ & "C:\Program Files (x86)\WiX Toolset v3.14\bin\light.exe" Product.wixobj
>>
Windows Installer XML Toolset Linker version 3.14.1.8722
Copyright (c) .NET Foundation and contributors. All rights reserved.

C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter\Product.wxs(8) : error LGHT0311 : A s
tring was provided with characters that are not available in the specified database code page '1252'. Either ch
ange these characters to ones that exist in the database's code page, or update the database's code page by mod
ifying one of the following attributes: Product/@Codepage, Module/@Codepage, Patch/@Codepage, PatchCreation/@Co
depage, or WixLocalization/@Codepage.
PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter>
```

Logramos crear el ejecutable Product.msi

```

NUEVO MSI CREADO EXITOSAMENTE
Tamaño: 31813632 bytes
Archivo: C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter\Product.msi

Archivos en la carpeta:

Directorio: C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter
r\PMEncrypter
r\PMEncrypter
r\PMEncrypter
r\PMEncrypter
r\PMEncrypter
r\PMEncrypter
r\PMEncrypter
r\PMEncrypter

r\PMEncrypter

r\PMEncrypter

r\PMEncrypter

```

(!) En este punto del trabajo, se nos complicó abrir/ejecutar correctamente el ejecutable, solo siendo posible la instalación de forma manual. Decidimos abordar la problemática más adelante.

```

PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter> Copy-Item "publish\PMEncrypter.exe" -Destination "C:\Program Files\PMEncrypter\PMEncrypter.exe" -Force
>
PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter> Test-Path "C:\Program Files\PMEncrypter\PMEncrypter.exe"
>
True
PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter> cd "C:\Program Files\PMEncrypter"
PS C:\Program Files\PMEncrypter> .\PMEncrypter.exe version
>
=== PMEncrypter - Herramienta de Hash SHA-256 ===

PMEncrypter v1.0.0
Herramienta de cálculo de hash SHA-256
Copyright 2024 - Proyecto de Empaquetado
PS C:\Program Files\PMEncrypter> .\PMEncrypter.exe hash "C:\Windows\System32\notepad.exe"
>
=== PMEncrypter - Herramienta de Hash SHA-256 ===

Archivo: notepad.exe
Tamaño: 360.448 bytes
SHA-256: 1552f6a579b77b61460df56cb4b2ce0a34fe96b6176829d7916275b806edc2bb
Ruta: C:\Windows\System32\notepad.exe
PS C:\Program Files\PMEncrypter>

```

Avanzamos creando el repositorio de nuestro proyecto en Github

```

>>
● PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter> git remote -v
>>
origin https://github.com/zkfakaii/PMEncrypter.git (fetch)
origin https://github.com/zkfakaii/PMEncrypter.git (push)
● PS C:\Users\dante\OneDrive\Documentos\mierdases\sist\PMEncrypter\PMEncrypter> git push -u origin main
>>
Enumerating objects: 250, done.
Counting objects: 100% (250/250), done.
Delta compression using up to 12 threads
Compressing objects: 100% (241/241), done.
Writing objects: 100% (250/250), 98.27 MiB | 7.89 MiB/s, done.
Total 250 (delta 91), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (91/91), done.
remote: warning: See https://gh.io/lfs for more information.
remote: warning: File publish/PMEncrypter.exe is 70.08 MB; this is larger than GitHub's recommended maximum
e size of 50.00 MB
remote: warning: GH001: Large files detected. You may want to try Git Large File Storage - https://git-lfs.g
ub.com.
To https://github.com/zkfakaii/PMEncrypter.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

Luego generamos nuestras carpetas para la estructura de Github Actions

Procediendo con el archivo del WorkFlow con release.yml

```

.github/workflows > release.yml
1  name: Build and Release PMEncrypter
2
3  on:
4    push:
5      tags:
6        - 'v*' # COn esta linea, se ejecuta cuando tu subes un tag que empiece con 'v'
7
8  jobs:
9    build:
10     runs-on: windows-latest
11
12     steps:
13       - name: Descargar código
14         uses: actions/checkout@v4
15
16       - name: Configurar .NET
17         uses: actions/setup-dotnet@v4
18         with:
19           dotnet-version: '6.0.x'
20
21       - name: Compilar proyecto
22         run: dotnet build --configuration Release
23
24       - name: Crear ejecutable
25         run: dotnet publish -c Release -r win-x64 --self-contained true -p:PublishSingleFile=true -o ./publish
26
27       - name: Generar checksum SHA-256
28         run: |
29           $hash = (Get-FileHash -Path "./publish/PMEncrypter.exe" -Algorithm SHA256).Hash
30           "SHA256: $hash" | Out-File -FilePath "./publish/checksum.txt" -Encoding UTF8
31           Get-Content "./publish/checksum.txt"
32
33       - name: Subir archivos como artefacto
34         uses: actions/upload-artifact@v4
35         with:
36           name: PMEncrypter-Files
37           path: |
38             publish/
39             retention-days: 1

```

```

using System;
using System.IO;
using System.Security.Cryptography;
using System.Text;

namespace PMEncrypter
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== PMEncrypter - Herramienta de Hash SHA-256 ===");
            Console.WriteLine();

            if (args.Length == 0)
            {
                MostrarAyuda();
                return;
            }

            string comando = args[0].ToLower();

            switch (comando)
            {
                case "hash":
                    if (args.Length < 2)
                    {
                        Console.WriteLine("Error: Debes especificar un archivo");
                        Console.WriteLine("Uso: PMEncrypter hash <ruta-del-archivo>");
                        return;
                    }
                    CalcularHash(args[1]);
                    break;

                case "version":
                    MostrarVersion();
                    break;

                case "help":
                case "--help":
                case "-h":
                    MostrarAyuda();
                    break;

                default:
                    Console.WriteLine($"Comando desconocido: {comando}");
                    MostrarAyuda();
                    break;
            }
        }
    }
}

```

```

static void CalcularHash(string filePath)
{
    try
    {
        if (!File.Exists(filePath))
        {
            Console.WriteLine($"Error: El archivo '{filePath}' no existe");
            return;
        }

        string hash = CalculateSHA256(filePath);
        FileInfo fileInfo = new FileInfo(filePath);

        Console.WriteLine($"Archivo: {Path.GetFileName(filePath)}");
        Console.WriteLine($"Tamaño: {fileInfo.Length:N0} bytes");
        Console.WriteLine($"SHA-256: {hash}");
        Console.WriteLine($"Ruta: {Path.GetFullPath(filePath)}");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error calculando hash: {ex.Message}");
    }
}

static string CalculateSHA256(string filePath)
{
    using SHA256 sha256 = SHA256.Create();
    using FileStream stream = File.OpenRead(filePath);

    byte[] hashBytes = sha256.ComputeHash(stream);

    StringBuilder sb = new StringBuilder();
    foreach (byte b in hashBytes)
    {
        sb.Append(b.ToString("x2"));
    }

    return sb.ToString();
}

static void MostrarVersion()
{
    Console.WriteLine("PMEncrypter v1.0.0");
    Console.WriteLine("Herramienta de cálculo de hash SHA-256");
    Console.WriteLine("Copyright 2024 - Proyecto de Empaquetado");
}

static void MostrarAyuda()
{
    Console.WriteLine("Uso: PMEncrypter <comando>");
    Console.WriteLine();
    Console.WriteLine("Comandos disponibles:");
    Console.WriteLine(" hash <archivo>   Calcular hash SHA-256 de un archivo");
    Console.WriteLine(" version          Mostrar información de versión");
    Console.WriteLine(" help            Mostrar esta ayuda");
    Console.WriteLine();
    Console.WriteLine("Ejemplos:");
    Console.WriteLine(" PMEncrypter hash documento.pdf");
    Console.WriteLine(" PMEncrypter version");
    Console.WriteLine(" PMEncrypter help");
}
}
}

```

¿Cómo usar?

- PMEncrypter tiene la funcionalidad de poder calcular el hash de un archivo en específico con el algoritmo de cifrado SHA-256, pudiendo también elegir el archivo a través de su ruta presente en el computador y finalmente teniendo la opción de poder ver los comandos disponibles a través del comando *help*, mostrando todos los comandos y acciones disponibles para el usuario (en este caso, *hash*, *version* y *help*).

Para poder calcular el hash de un archivo, se debe hacer lo siguiente:

- **(Opcional)** Mover el archivo en la carpeta donde se encuentra la aplicación usada para el cálculo de hash (esto es más por comodidad, sin embargo, cualquier archivo que tenga su ruta disponible sirve).
- Correr la aplicación de cálculo de hash en CMD o en PowerShell.
- Introducir el comando "`PMEncrypter.exe hash <ruta de archivo>`"
- Esperar a que el algoritmo termine de calcular el hash.
- Una vez terminado, el hash será mostrado en la consola.

Ejemplos sencillos de uso

- **Calcular hash SHA-256 de un archivo:**

- Resultado esperado:

```
=== PMEncrypter - Herramienta de Hash SHA-256 ===
```

```
Archivo: test.txt
```

```
Tamaño: 62 bytes
```

```
SHA-256: 4c00aa3ed3b1f66c07e25bb3518f1c5cdf103d1c42db207a88a0716470f98419
```

```
Ruta: C:\Users\Darryl\PMEncrypter\test.txt
```

- **Revisar versión del programa:**

- Resultado esperado:

```
PMEncrypter v1.0.0
```

```
Herramienta de cálculo de hash SHA-256
```

```
Copyright 2025 - Proyecto de Empaquetado
```

- **Comando de ayuda:**

- Resultado esperado:

```
Comandos disponibles:
```

```
hash <archivo>    Calcular hash SHA-256 de un archivo
```

```
version           Mostrar información de versión
```

```
help              Mostrar esta ayuda
```

```
Ejemplos:
```

```
PMEncrypter hash documento.pdf
```

```
PMEncrypter version
```

```
PMEncrypter help
```

API detallado

- **Comando:** `hash <archivo>`
 - **Parámetros presentes en el comando:**
 - `<archivo>`
La ruta del archivo a calcular, debe de ser legible y existente.
 - **Salida estándar / esperada:**
 - Nombre del archivo elegido
 - Tamaño en bytes
 - Código hash SHA-256 en hexadecimal
 - Ruta absoluta del archivo
- **Comando:** `help`
 - Muestra los comandos disponibles y usables en la aplicación, junto a ejemplos simples de cómo usarlos.
- **Comando:** `version`
 - Muestra la versión actual de la aplicación junto a otros detalles.

Formato de archivo

- El formato de archivo que usa la aplicación, al menos visto en consola es `.sha256`, el cual usualmente se ve de la siguiente manera:

```
- <hash-hex> (hash-calculado) <nombre-archivo>.sha256
```

Consideraciones y limitaciones de seguridad

- **La aplicación no protege datos, solo calcula hashes:**
 - Este programa **NO** se debe usar como encriptador de archivos, ya que este solo tiene la funcionalidad de calcular solo el hash de un archivo en específico.
- **SHA-256 es resistente, pero no infalible:**
 - No se debe usar el algoritmo SHA-256 en caso de querer seguridad a largo plazo, pese a que este no se pueda descriptar tan rápidamente.
- **Los hashes NO se deben usarse para contraseñas:**
 - Los hashes no poseen la funcionalidad de ser contraseñas fiables a largo plazo debido a carecer de dos factores importantes, siendo estos:
 - 1) La carencia del uso de un algoritmo resistente a métodos de fuerza bruta, como Argon2, bcrypt, entre otros.
 - 2) La carencia del uso del proceso de aplicar “sal” o coste computacional para poder ser una contraseña segura.
- **Riesgos a tomar en cuenta al cifrar archivos de muchos GB:**
 - Se recomienda **NO** usar la aplicación para calcular el hash de archivos medidos en GB, debido a poder enfrentar las siguientes complicaciones:
 - 1) Uso significativo de la CPU y/o RAM presente en la computadora, lo cual en el mejor de los casos no será notable, y en el peor, congelará todos los procesos de la computadora.
 - 2) Tiempos de espera sumamente largos, los cuales deja la aplicación unusable hasta que termine el cálculo.
 - 3) Archivos bloqueados si son necesitados por otros procesos externos al de la aplicación.

Derivación y manejo de claves

- **¿Qué es?**
 - La derivación de claves es el proceso usado para convertir una contraseña, una frase secreta o una clave base en una clave criptográfica segura utilizada para poder cifrar datos de manera segura, usualmente generados tras funciones como los antes mencionados Argon2 o bcrypt, los cuales aplican miles (o millones) de iteraciones, agregan sal y hacen que una clave sea muchísima más lenta de ser obtenida a través de métodos de fuerza bruta.
- **¿Cómo se relaciona al proyecto?**
 - Mientras que lo antes mencionado no se aplica de manera directa a la aplicación, se debe de resaltar que PMEncrypter **NO** es un generador de contraseñas seguras, solo es una aplicación de cálculo de hash.
- **Prácticas recomendadas sobre el manejo de claves**
 - No guardar claves en texto plano.
 - Evitar contraseñas cortas o predecibles.
 - Usar sal única para cada contraseña.
 - No reutilizar contraseñas.
 - Nunca imprimir contraseñas o claves por logs o por la consola.
 - Limpiar las contraseñas de la memoria cuán pronto sea posible.

Referencia CLI

- `hash` — Calcula el hash del archivo seleccionado.
- `version` — Muestra la versión de la aplicación.
- `help` || `--help` || `-h` — Ayuda, muestra los dos comandos antes mencionados.
- En caso de no introducir algún comando definido en la aplicación, se arrojará el mensaje de `Comando desconocido: (comando no-definido)`.

Opciones comunes

- `hash <ruta-del-archivo>` — Parámetro necesario para que el comando de hash pueda funcionar correctamente, en caso de no tener, el comando no se ejecutará y arrojará un error.

Guía de testeo

Test de ejecución (ejecutado desde CLI):

1. Preparaciones:

- 1.1. Se debe de poseer el archivo PMEncrypter.exe o en otro caso, tener compilado el proyecto en Visual Studio.
- 1.2. Tener a mano algunos archivos de prueba, se recomienda que sean archivos de texto debido a la facilidad de manejar estos y sus pesos en bytes (también se pueden crear en la misma carpeta donde se encuentra el ejecutable para mayor accesibilidad, se recomienda tener un archivo de texto con contenido dentro y otro vacío)

2. Tests del comando de hash (*hash <archivo>*):

2.1. Archivo con datos (como un archivo de texto con contenido dentro)

- 2.1.1. Debe de aparecer en la consola lo siguiente:

Archivo: <nombre de archivo>
Tamaño: <tamaño de archivo>
SHA-256: <valor SHA-256 del archivo>
Ruta: <ruta del archivo>

2.2. Archivo sin datos (Archivo de texto sin contenido)

- 2.2.1. Debe de arrojar el siguiente código hash:
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b78
52b855

El cual es comúnmente visto en archivos vacíos.

2.3. Archivo inexistente:

- 2.3.1. Debe de arrojar un error de archivo no encontrado o no especificado, lo cual acaba por terminar el proceso del programa

3. Test de otros comandos misceláneos:

3.1. Test de comando help:

3.1.1. Debe de escribir en la consola lo siguiente:

Uso: PMEncrypter <comando>

Comandos disponibles:

hash <archivo> Calcular hash SHA-256 de un archivo

version Mostrar información de esta versión

help Mostrar esta ayuda

Ejemplos:

PMEncrypter hash documento.pdf

PMEncrypter version

PMEncrypter help

3.2. Test de comando version:

3.2.1. Debe de escribir en la consola lo siguiente:

PMEncrypter v1.0.0

Herramienta de cálculo de hash SHA-256

Copyright 2025 - Proyecto de Empaquetado

4. Recomendaciones al testear:

4.1. Siempre revisar que el hash calculado y recibido de la aplicación concuerde y sea corroborado con otros programas los cuales puedan usar SHA-256 también.

4.2. Probar archivos pequeños y medianos, *no se recomienda mucho los archivos grandes debido a las posibles complicaciones antes mencionadas.*

4.2.1. Pero en caso de probar con un archivo grande, ver si el programa no se cierra durante el proceso y pueda mostrar el resultado correctamente.

4.3. Procurar que todos los errores se muestran en la consola y que no causen que el programa se cierre de golpe.

Registro de cambios y compatibilidad

- **v1.0.0:** Lanzamiento oficial, solo compatible con Windows debido a ser construido en base a **.NET 10**, el cual no posee características que permitan el uso del programa en otros sistemas operativos como Linux o macOS.

Reconocimientos y referencias

- ChatGPT: Ayuda con el código, consejos y con ciertos bugs.